

ThinkBeforeClick

Technology Stack Overview — what each tool does in this project (max 2 lines each)

Backend & API

Technology	What it does in this project
Python	The core programming language for the entire backend, all detection logic, and the ML pipeline.
FastAPI	Web framework serving the /analyze, /health, and /report endpoints. Chosen for built-in request validation and auto-generated docs (/docs).
Uvicorn	The ASGI server that actually runs the FastAPI application, both locally and in production on Render.
Pydantic	Defines and validates the exact shape of every request/response (AnalyzeRequest, AnalyzeResponse) so bad input is rejected before it reaches the logic.

Data Storage

Technology	What it does in this project
SQLite	Lightweight, file-based database storing scan and report records — no separate DB server needed.
SQLAlchemy	Python ORM layer connecting FastAPI to SQLite via the Scan and Report table models.

Detection Layer 1 — Rules

Technology	What it does in this project
re (Python regex)	Powers the rule engine's keyword and URL-structure detection, including obfuscation-aware matching (e.g. "u.r.g.e.n.t").

Detection Layer 2 — Machine Learning

Technology	What it does in this project
scikit-learn (1.7.2)	Provides TF-IDF vectorization and the Logistic Regression classifier that scores messages for phishing likelihood.
joblib	Saves and loads the trained classifier and vectorizer (classifier.joblib, vectorizer.joblib) so the model doesn't need retraining on every server start.
pandas	Loads and cleans the training dataset (CSV) before it's fed into scikit-learn for training.

Detection Layer 3 — Threat Intelligence

Technology	What it does in this project
Google Safe Browsing API v4	Checks submitted URLs against Google's live database of known malicious/malware sites — an independently-verified external signal.
python-whois	Looks up a URL's domain registration date to flag newly-registered domains, a common phishing indicator.
requests	Handles the HTTP calls to the Safe Browsing API from within the threat-intel service.

Frontend

Technology	What it does in this project
HTML / CSS / JavaScript	Builds the entire analyzer web app with no framework — intentionally lightweight and dependency-free.
HTML5 Canvas	Renders the continuous matrix-rain background animation behind every stage of the UI.
GSAP (self-hosted)	Drives the timed stage sequencing — intro glitch, terminal-panel reveal, verdict card entrance. Self-hosted after a CDN version silently broke animations.
Fetch API	Sends the user's message/URL from the browser to the backend's /analyze endpoint and receives the verdict.

Infrastructure & Deployment

Technology	What it does in this project
GitHub	Hosts the source code; every push triggers an automatic redeploy on Render.
Render (Web Service)	Hosts and runs the live FastAPI backend, including environment variables like the Safe Browsing API key.
Render (Static Site)	Serves the plain HTML/CSS/JS frontend with no build step required.
Custom Domain (thinkbeforeclick.me)	Points a human-readable domain at the deployed Render frontend instead of a default Render subdomain.
CORS (FastAPI Middleware)	Restricts which websites are allowed to call the backend API — locked to the deployed frontend domain, not left open to the whole internet.